CCT College Dublin

Assessment Cover Page

Module Title:	**Algorithms & Constructs (HDip in Computing - Sep 2025 cohort)**
Assessment Title:	CA 2 - Report
Lecturer Name:	Taufique Ahmed
Student Full Name:	Alexsandro da Silva Saraiva
Student Number:	2025776
Assessment Due Date:	Sunday, Due 23 May 2026 @ 23:59
Date of Submission:	Thursday, 21 May 2026

Declaration

By submitting this assessment, I confirm that I have read the CCT policy on Academic Misconduct and understand the implications of submitting work that is not my own or does not appropriately reference material taken from a third party or other source. I declare it to be my own work and that all material from third parties has been appropriately referenced. I further confirm that this work has not previously been submitted for assessment by myself or someone else in CCT College Dublin or any other higher education institution.

Organization Choice: Bank

A bank is an ideal choice because it has a clearly defined hierarchical management structure like Head Manager, Senior Manager, Assistant, Team Lead etc. and distinct departments like Customer Servic, IT Development, etc. with strict operational protocols that naturally lead themselves into something like a tree-based hierarchy. Banks operate in a regulated environment where role clarity, data integrity, and structured record-keeping are critical making sorting, searching, and validated input essential real-world concerns.

Algorithm Justifications

Sorting with Merge Sort (Recursive)

I selected Merge Sort because the applicant data originates from a file of unknown ordering, if any and guarantees O(n log n) worst-case performance regardless of the initial arrangement. Unlike Quick Sort, whose worst case degrades to $O(n^2)$ on already sorted or nearly sorted inputs (if the file is partially alphabetized), merge sort is consistently efficient. It is also a stable, meaning employees sharing the same last name retain their original relative order which is important for data provenance in a banking context. Compared to Heap Sort which is also O(n log n) worst-case, merge sort's stability and naturally recursive divide-and-conquer structure make it the fitter for this assignment's requirement of a recursive algorithm.

Searching with Binary Search (Recursive)

I selected Recursive Binary Search because the list is already sorted (a precondition we satisfy using merge sort before search), allowing binary search to locate any name in O(log n) time. A linear search would require O(n) comparisons on every query which is unacceptable as the employees list grows. Binary search's recursive satisfies the assignment's recursive requirement with minimal overhead. Binary Search provides the optimal balance of speed, simplicity, and compatibility with our sorted data structure.

Both algorithms were implemented recursively, with case-insensitive string comparison to ensure consistent behavior regardless of how names are capitalized during input or file reading.

Project's GitHub link:

https://github.com/asilvasaraiva/cct-ca2-2025776